

Разработка A/B-тестирования и анализ результатов

Вы работаете продуктовым аналитиком в компании, которая разрабатывает развлекательное приложение с функцией «бесконечной» ленты, как, например, в приложениях с короткими видео. В вашем приложении существует две модели монетизации: первая — ежемесячная платная подписка, которая позволяет пользователям смотреть ленту без рекламы, вторая — демонстрация рекламы для пользователей, которые ещё не оформили подписку.

Команда разработчиков рекомендательных систем создала новый алгоритм рекомендаций, который, по их мнению, будет показывать более интересный контент для каждого пользователя. Вас, как аналитика, просят помочь рассчитать параметры A/B-теста, который позволит проверить эту гипотезу, и проанализировать его результаты.

Описание данных

Вы будете работать с тремя таблицами:

- `sessions_project_history.csv` — таблица с историческими данными по сессиям пользователей на период с 2025-08-11 по 2025-09-23. Путь к файлу: `/datasets/sessions_project_history.csv`.
- `sessions_project_test_part.csv` — таблица с данными за первый день проведения A/B-теста, то есть за 2025-10-14. Путь к файлу: `/datasets/sessions_project_test_part.csv`.
- `sessions_project_test.csv` — таблица с данными за весь период проведения A/B-теста, то есть с 2025-10-14 по 2025-11-02. Путь к файлу: `/datasets/sessions_project_test.csv`.

У этих таблиц почти совпадает структура и содержание колонок, различаются лишь периоды наблюдения.

Поля таблиц `sessions_project_history.csv`, `sessions_project_test.csv`, `sessions_project_test_part.csv`:

- `user_id` — идентификатор пользователя;
- `session_id` — идентификатор сессии в приложении;
- `session_date` — дата сессии;
- `session_start_ts` — дата и время начала сессии;
- `install_date` — дата установки приложения;
- `session_number` — порядковый номер сессии для конкретного пользователя;

- `registration_flag` — является ли пользователь зарегистрированным;
- `page_counter` — количество просмотренных страниц во время сессии;
- `region` — регион пользователя;
- `device` — тип устройства пользователя;
- `test_group` — тестовая группа (в таблице с историческими данными этого столбца нет).

Что нужно сделать

Ваши задачи: рассчитать параметры теста, оценить корректность его проведения и проанализировать результаты эксперимента.

1. Работа с историческими данными (EDA)

1.1. Загрузка исторических данных

На первом этапе поработайте с историческими данными приложения:

- Импортируйте библиотеку `pandas`.
- Считайте и сохраните в датафрейм `sessions_history` CSV-файл с историческими данными о сессиях пользователей `sessions_project_history.csv`.

Выведите на экран первые пять строк полученного датафрейма.

```
In [1]: # Импортируем библиотеки
import pandas as pd
import numpy as np

# Загружаем библиотеки для визуализации данных
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: # Выгружаем данные в переменную df
sessions_history = pd.read_csv('/datasets/sessions_project_history.csv')
```

```
In [3]: # Выводим первые строки датафрейма на экран
sessions_history.head()
```

```
Out[3]:
```

	<code>user_id</code>	<code>session_id</code>	<code>session_date</code>	<code>session_start_ts</code>	<code>install_date</code>	<code>session_number</code>
0	E302123B7000BFE4	F9AF61A0C2023832	2025-08-15	2025-08-15 17:47:35	2025-08-15	1
1	2530F72E221829FB	85003A206CBDAC6F	2025-08-15	2025-08-15 16:42:14	2025-08-15	1
2	876E020A4FC512F5	3677423E49D72DEE	2025-08-15	2025-08-15 12:30:00	2025-08-15	1

	user_id	session_id	session_date	session_start_ts	install_date	session_number
3	2640B349E1D81584	956B45F5915CA225	2025-08-15	2025-08-15 15:31:31	2025-08-15	1
4	94E1CBFAEF1F5EE9	83BF0DA35F9F1F40	2025-08-15	2025-08-15 21:33:53	2025-08-15	1

In [4]:

```
# Выводим информацию о датафрейме
sessions_history.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 435924 entries, 0 to 435923
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   user_id                435924 non-null object
1   session_id             435924 non-null object
2   session_date           435924 non-null object
3   session_start_ts       435924 non-null object
4   install_date           435924 non-null object
5   session_number         435924 non-null int64
6   registration_flag      435924 non-null int64
7   page_counter           435924 non-null int64
8   region                 435924 non-null object
9   device                 435924 non-null object
dtypes: int64(3), object(7)
memory usage: 33.3+ MB
```

In [5]:

```
# Приводим столбцы 'session_start_ts', 'session_date', 'install_date' к munny date
sessions_history['session_start_ts'] = pd.to_datetime(sessions_history['session_start_ts'])
sessions_history['session_date'] = pd.to_datetime(sessions_history['session_date']).
sessions_history['install_date'] = pd.to_datetime(sessions_history['install_date']).
sessions_history.head()
```

Out[5]:

	user_id	session_id	session_date	session_start_ts	install_date	session_number
0	E302123B7000BFE4	F9AF61A0C2023832	2025-08-15	2025-08-15 17:47:35	2025-08-15	1
1	2530F72E221829FB	85003A206CBDAC6F	2025-08-15	2025-08-15 16:42:14	2025-08-15	1
2	876E020A4FC512F5	3677423E49D72DEE	2025-08-15	2025-08-15 12:30:00	2025-08-15	1
3	2640B349E1D81584	956B45F5915CA225	2025-08-15	2025-08-15 15:31:31	2025-08-15	1
4	94E1CBFAEF1F5EE9	83BF0DA35F9F1F40	2025-08-15	2025-08-15 21:33:53	2025-08-15	1

1.2. Знакомство с данными

- Для каждого уникального пользователя `user_id` рассчитайте количество уникальных сессий `session_id`.
- Выведите на экран все данные из таблицы `sessions_history` для одного пользователя с наибольшим количеством сессий. Если таких пользователей несколько,

выберите любого из них.

- Изучите таблицу для одного пользователя, чтобы лучше понять логику формирования каждого столбца данных.

In [6]:

```
# Считаем количество уникальных сессий 'session_id' для каждого уникального пользова
user_session_count = sessions_history.groupby('user_id')['session_id'].nunique()
# Считаем максимальное количество сессий на одного игрока
max_session_user = user_session_count.idxmax()
max_session_count = user_session_count.max()
# Выводим информацию о пользователе с максимальным количеством сессий
print(f"Пользователь с максимальным количеством сессий:{max_session_user}, количеств
session_max_user = sessions_history[sessions_history['user_id']==max_session_user]
# Выводим информацию о датафрейме
print(session_max_user.info())
# Выводим первые строки датафрейма на экран
print(session_max_user.head())
```

Пользователь с максимальным количеством сессий:10E0DEFC1ABDBBE0, количество сессий: 10

<class 'pandas.core.frame.DataFrame'>

Int64Index: 10 entries, 115558 to 414743

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	user_id	10 non-null	object
1	session_id	10 non-null	object
2	session_date	10 non-null	object
3	session_start_ts	10 non-null	datetime64[ns]
4	install_date	10 non-null	object
5	session_number	10 non-null	int64
6	registration_flag	10 non-null	int64
7	page_counter	10 non-null	int64
8	region	10 non-null	object
9	device	10 non-null	object

dtypes: datetime64[ns](1), int64(3), object(6)

memory usage: 880.0+ bytes

None

	user_id	session_id	session_date	session_start_ts	\
115558	10E0DEFC1ABDBBE0	B8F0423BBFFCF5DC	2025-08-14	2025-08-14 13:57:39	
191751	10E0DEFC1ABDBBE0	87CA2FA549473837	2025-08-15	2025-08-15 16:42:10	
239370	10E0DEFC1ABDBBE0	4ADD8011DCDCE318	2025-08-16	2025-08-16 19:53:21	
274629	10E0DEFC1ABDBBE0	DF0FD0E09BF1F3D7	2025-08-17	2025-08-17 15:03:43	
302501	10E0DEFC1ABDBBE0	3C221774B4DE6885	2025-08-18	2025-08-18 17:29:14	

	install_date	session_number	registration_flag	page_counter	region	\
115558	2025-08-14	1	0	4	CIS	
191751	2025-08-14	2	0	3	CIS	
239370	2025-08-14	3	0	3	CIS	
274629	2025-08-14	4	0	1	CIS	
302501	2025-08-14	5	0	4	CIS	

	device
115558	Android
191751	Android
239370	Android
274629	Android
302501	Android

1.3. Анализ числа регистраций

Одна из важнейших метрик продукта — число зарегистрированных пользователей.

Используя исторические данные, визуализируйте, как менялось число регистраций в приложении за время его существования.

- Агрегируйте исторические данные и рассчитайте число уникальных пользователей и число зарегистрированных пользователей для каждого дня наблюдения. Для простоты считайте, что у пользователя в течение дня бывает одна сессия максимум и статус регистрации в течение одного дня не может измениться.
- Постройте линейные графики общего числа пользователей и общего числа зарегистрированных пользователей по дням. Отобразите их на одном графике.
- Постройте отдельный линейный график доли зарегистрированных пользователей от всех пользователей по дням.
- На обоих графиках должны быть заголовок, подписанные оси X и Y, сетка и легенда.

In [7]:

```
# Считаем общее количество уникальных пользователей для каждого дня
daily_user = sessions_history.groupby('session_date')['user_id'].nunique().rename('total_users')
# Считаем общее количество уникальных зарегистрированных пользователей для каждого дня
daily_registered = sessions_history[sessions_history['registration_flag'] == 1].groupby('session_date')['user_id'].nunique().rename('registered_users')
# Объединим в один датафрейм
daily = pd.concat([daily_user, daily_registered], axis=1).fillna(0).astype(int)
daily = daily.sort_index()

# Строим график
plt.figure(figsize=(12, 6))
plt.plot(daily.index, daily['total_users'], label = 'Общее количество уникальных пользователей')
plt.plot(daily.index, daily['registered_users'], label = 'Общее количество зарегистрированных пользователей')
# Настраиваем оформление графика
plt.title('Динамика общего количества уникальных и зарегистрированных пользователей')
plt.xlabel('Дата')
plt.ylabel('Количество пользователей')
# Добавляем сетку графика
plt.grid()
# Добавляем легенду
plt.legend()
# Градус вращения подписи по оси X
plt.xticks(rotation=45)
# Выводим график
plt.show()
# Выводим первые строки датафрейма на экран
print(daily.head())
```



session_date	total_users	registered_users
2025-08-11	3919	169
2025-08-12	6056	336
2025-08-13	8489	464
2025-08-14	10321	625
2025-08-15	14065	840

In [8]:

```
# Считаем долю зарегистрированных пользователей от всех пользователей по дням
daily['registered_share'] = daily['registered_users'] / daily['total_users']
daily['registered_share'] = daily['registered_share'].fillna(0)

# Строим график
plt.figure(figsize=(12, 6))
plt.plot(daily.index, daily['registered_share'], label = 'Доля зарегистрированных по
# Настраиваем оформление графика
plt.title('Доля зарегистрированных пользователей по дням')
plt.xlabel('Дата')
plt.ylabel('Доля зарегистрированных пользователей')
# Добавляем сетку графика
plt.grid()
# Добавляем легенду
plt.legend()
# Градус вращения подписи по оси X
plt.xticks(rotation=45)
# Выводим график
plt.show()
# Выводим первые строки датафрейма на экран
print(daily[['total_users', 'registered_users', 'registered_share']].head())
```



session_date	total_users	registered_users	registered_share
2025-08-11	3919	169	0.043123
2025-08-12	6056	336	0.055482
2025-08-13	8489	464	0.054659
2025-08-14	10321	625	0.060556
2025-08-15	14065	840	0.059723

1.4. Анализ числа просмотренных страниц

Другая важная метрика продукта — число просмотренных страниц в приложении. Чем больше страниц просмотрено, тем сильнее пользователь увлечён контентом, а значит, выше шансы, что он зарегистрируется и оплатит подписку.

- Найдите количество сессий для каждого значения количества просмотренных страниц. Например: одну страницу просмотрели в 29 160 сессиях, две страницы — в 105 536 сессиях и так далее.
- Постройте столбчатую диаграмму, где по оси X будет число просмотренных страниц, по оси Y — количество сессий.
- На диаграмме должны быть заголовок, подписанные оси X и Y.

In [9]:

```
# Считаем количество сессий для каждого значения количества просмотренных страниц
page_session = sessions_history['page_counter'].value_counts().sort_index()
# Выводим первые строки датафрейма на экран
print('Количество сессий для каждого значения количества просмотренных страниц')
print(page_session.head(10))

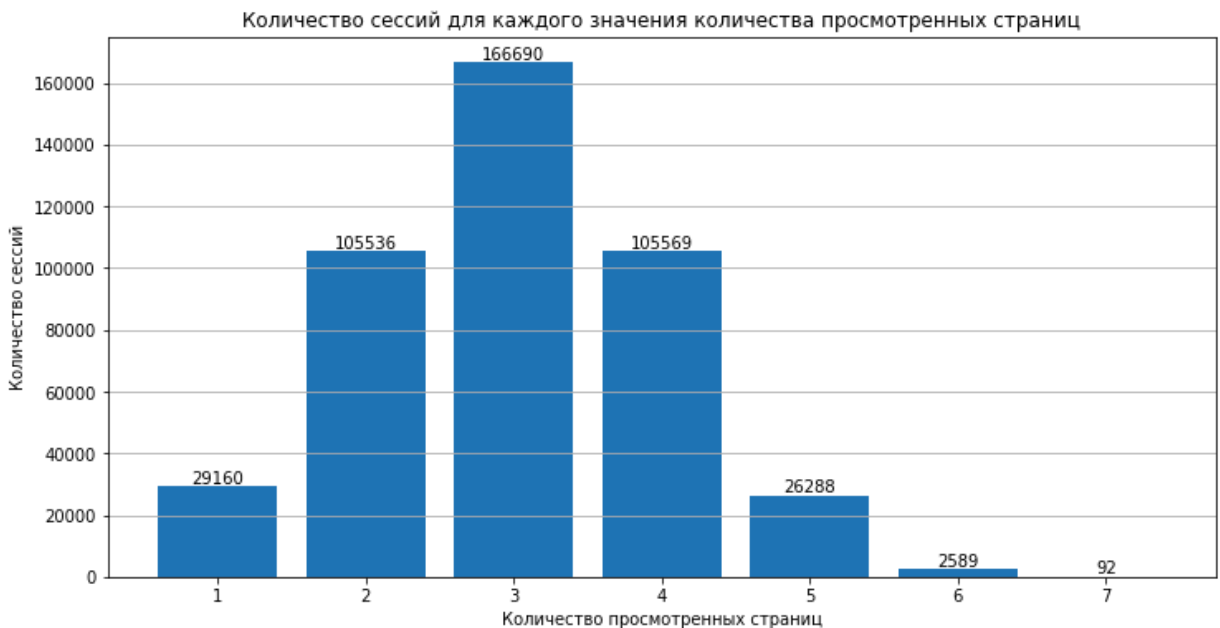
# Строим график
plt.figure(figsize=(12, 6))
plt.bar(page_session.index, page_session.values)
# Настраиваем оформление графика
plt.title('Количество сессий для каждого значения количества просмотренных страниц')
plt.xlabel('Количество просмотренных страниц')
plt.ylabel('Количество сессий')
# Добавляем подписи значений над столбцами
for i, v in enumerate(page_session.values):
```

```
plt.text(page_session.index[i], v, str(v), ha='center', va='bottom')
# Добавляем сетку графика
plt.grid(axis='y')
# Выводим график
plt.show()
```

Количество сессий для каждого значения количества просмотренных страниц

```
1    29160
2   105536
3   166690
4   105569
5    26288
6     2589
7         92
```

Name: page_counter, dtype: int64



1.5. Доля пользователей, просмотревших более четырёх страниц

Продуктовая команда продукта считает, что сессии, в рамках которых пользователь просмотрел 4 и более страниц, говорят об удовлетворённости контентом и алгоритмами рекомендаций. Этот показатель является важной прокси-метрикой для продукта.

- В датафрейме `sessions_history` создайте дополнительный столбец `good_session`. В него войдёт значение `1`, если за одну сессию было просмотрено 4 и более страниц, и значение `0`, если было просмотрено меньше.
- Постройте график со средним значением доли успешных сессий от всех сессий по дням за весь период наблюдения.

In [10]:

```
# Создаем дополнительный столбец good_session
sessions_history['good_session'] = (sessions_history['page_counter'] >= 4).astype(int)
daily_good_session = sessions_history.groupby('session_date')['good_session'].mean()

# Считаем среднее значение за весь период
mean_all = daily_good_session.mean()

# Считаем среднее значение за последний период (например, последние 7 дней)
mean_last_period = daily_good_session.tail(7).mean()
```

```

# Считаем стандартное отклонение
std_dev = daily_good_session.std()

# Строим график
plt.figure(figsize=(12, 6))
plt.plot(daily_good_session.index, daily_good_session.values, label = 'Доля успешных')
# Добавляем линию среднего значения
plt.axhline(mean_all, color='red', linestyle='--', label="Среднее значение")

# Настраиваем оформление графика
plt.title('Доля успешных сессий от всех сессий по дням за весь период наблюдения')
plt.xlabel('Дата')
plt.ylabel('Доля успешных сессий')

# Добавляем сетку графика
plt.grid()

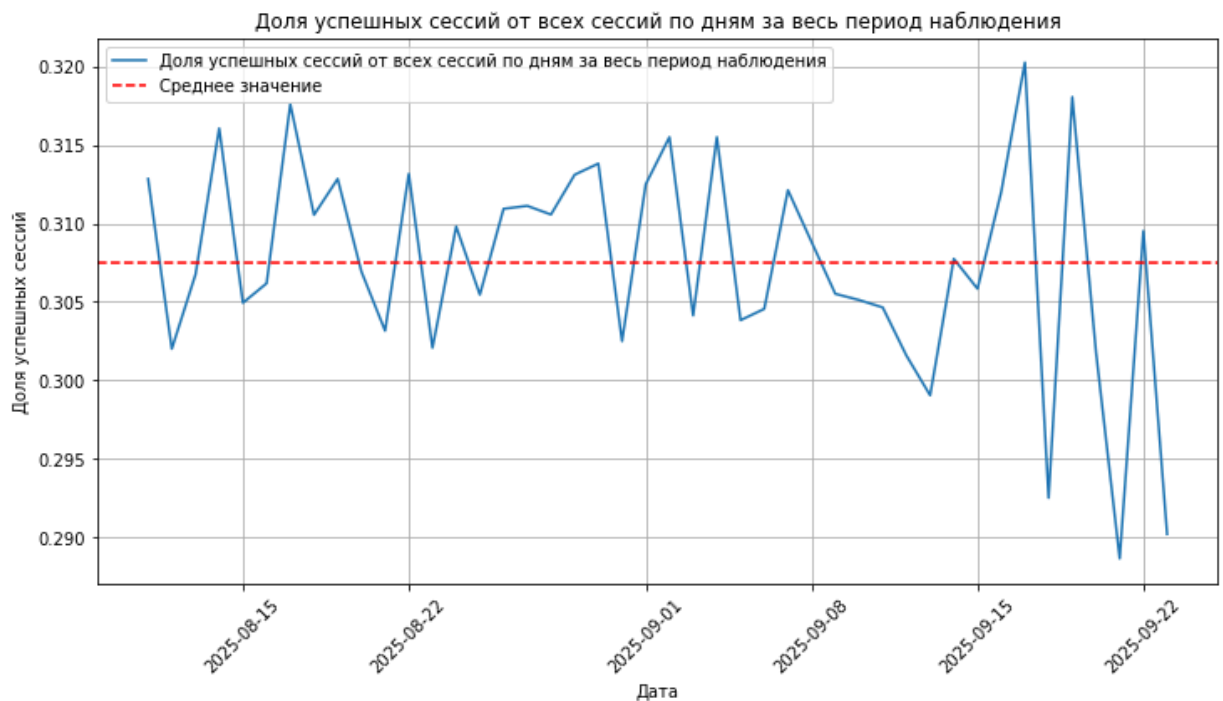
# Добавляем легенду
plt.legend()

# Градус вращения подписи по оси X
plt.xticks(rotation=45)

# Выводим график
plt.show()

# Выводим первые строки датафрейма на экран
print("Доля успешных сессий от всех сессий по дням за весь период наблюдения:")
print(daily_good_session.head())
print(f"Среднее за весь период: {mean_all}")
print(f"Среднее за последний период: {mean_last_period}")
print(f"Стандартное отклонение: {std_dev}")

```



Доля успешных сессий от всех сессий по дням за весь период наблюдения:

```

session_date
2025-08-11    0.312835
2025-08-12    0.302015
2025-08-13    0.306750
2025-08-14    0.316055
2025-08-15    0.304941
Name: good_session, dtype: float64
Среднее за весь период: 0.3075441043954679

```

2. Подготовка к тесту

При планировании теста необходимо проделать несколько важных шагов:

- Сформулировать нулевую и альтернативную гипотезы
- Определиться с целевой метрикой.
- Рассчитать необходимый размер выборки.
- Исходя из текущих значений трафика рассчитать необходимую длительность проведения теста.

2.1 Формулировка нулевой и альтернативной гипотез

Перед тем как проводить A/B-тест, необходимо сформулировать нулевую и альтернативную гипотезы. Напомним изначальное условие: команда разработчиков рекомендательных систем создала новый алгоритм, который, по их мнению, будет показывать более интересный контент для каждого пользователя.

О какой метрике идёт речь? Как она будет учтена в формулировке гипотез?

Сформулируйте нулевую и альтернативную гипотезы:

1. Так как разработчикам важно разработать развлекательное приложение с функцией «бесконечной» ленты, как, например, в приложениях с короткими видео, то ему в первую очередь важны такие показатели как, длительность нахождения в приложении и просмотр страниц. Из этого следует метрика `page_counter` - количество просмотренных страниц во время сессии.
 2. Формулируем гипотезу:
 - Нулевая гипотеза предполагает отсутствие различий выборочных характеристик между группами наблюдений. В терминах A/B-тестирования это была бы гипотеза о том, что целевая метрика не различается между группами A и B: $H_0: \mu_A = \mu_B$. По показателю `page_counter` новый алгоритм не влияет на количество просмотренных страниц в рамках сессии. За показатели возьмем μ_A -среднее число просмотренных страниц по старому алгоритму, μ_B -среднее число просмотренных страниц по новому алгоритму.
 - Альтернативная гипотеза предполагает существование различий выборочных характеристик между группами наблюдений. В терминах A/B-тестирования это была бы гипотеза о том, что целевая метрика различается в группах A и B. Гипотеза: $H_1: \mu_A > \mu_B$. По показателю `page_counter` новый алгоритм влияет на количество просмотренных страниц в рамках сессии, происходит увеличение. За показатели возьмем μ_A -среднее число просмотренных страниц по старому алгоритму, μ_B -среднее число просмотренных страниц по новому алгоритму.
- Итог: метрика - `page_counter` (количество просмотренных страниц в рамках сессии), тип теста - односторонний, анализ проводится в рамках прироста показателей.

2.2. Расчёт размера выборки

В рамках курса вы уже рассчитывали размеры выборки и использовали для этого онлайн-калькулятор. В этом задании предлагаем воспользоваться готовым кодом и рассчитать необходимое для вашего эксперимента количество пользователей.

Для этого установите в коде ниже следующие параметры:

- Уровень значимости — 0.05.
- Вероятность ошибки второго рода — 0.2.
- Мощность теста.
- Минимальный детектируемый эффект, или MDE, — 3%. Обратите внимание, что здесь нужно указать десятичную дробь, а не процент.

При расчёте размера выборки используйте метод `solve_power()` из класса `power.NormalIndPower` модуля `statsmodels.stats`.

Запустите ячейку и изучите полученное значение.

```
In [11]: from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize

# Задаём параметры
alpha = 0.05 # Уровень значимости
beta = 0.2 # Ошибка второго рода, часто 1 - мощность
power = 1-beta # Мощность теста
p = 0.3 # Базовый уровень доли
mde = 0.03 * p # Минимальный детектируемый эффект
p1 = p
p2 = p1 + mde
effect_size = proportion_effectsize(p1, p1 + mde)

# Инициализируем класс NormalIndPower
power_analysis = NormalIndPower()

# Рассчитываем размер выборки
sample_size = power_analysis.solve_power(
    effect_size = effect_size,
    power = power,
    alpha = alpha,
    ratio = 1 # Равномерное распределение выборок
)

print(f"Необходимый размер выборки для каждой группы: {int(sample_size)}")
```

Необходимый размер выборки для каждой группы: 41040

2.3. Расчёт длительности А/В-теста

Используйте данные о количестве пользователей в каждой выборке и среднем количестве пользователей приложения. Рассчитайте длительность теста, разделив одно на другое.

- Рассчитайте среднее количество уникальных пользователей приложения в день.
- Определите длительность теста исходя из рассчитанного значения размера выборок и среднего дневного трафика приложения. Количество дней округлите в большую сторону.

In [12]:

```
from math import ceil

# Среднее количество пользователей приложения в день по историческим данным
avg_daily_users = daily_user.mean()

# Рассчитываем длительность теста в днях как отношение размера выборки к среднему числу
test_duration = ceil((sample_size*2)/avg_daily_users)

print(f"Рассчитанная длительность A/B-теста при текущем уровне трафика в {avg_daily_users} пользователей в день составит {test_duration} дней")
```

Рассчитанная длительность A/B-теста при текущем уровне трафика в 9907.363636363636 пользователей в день составит 9 дней

3. Мониторинг A/B-теста

3.1. Проверка распределения пользователей

A/B-тест успешно запущен, и уже доступны данные за первые три дня. На этом этапе нужно убедиться, что всё идёт хорошо: пользователи разделены правильным образом, а интересующие вас метрики корректно считаются.

- Считайте и сохраните в датафрейм `sessions_test_part` CSV-файл с историческими данными о сессиях пользователей `sessions_project_test_part.csv`.
- Рассчитайте количество уникальных пользователей в каждой из экспериментальных групп для одного дня наблюдения.
- Рассчитайте и выведите на экран процентную разницу в количестве пользователей в группах A и B. Постройте любую удобную визуализацию, на которой будет видно возможное различие двух групп.

Для расчёта процентной разницы воспользуйтесь формулой:

$$P = 100 \cdot \frac{|A - B|}{A}$$

In [13]:

```
# Импортируем библиотеки
import pandas as pd
import numpy as np

# Загружаем библиотеки для визуализации данных
import matplotlib.pyplot as plt
import seaborn as sns
```

In [14]:

```
# Выгружаем данные в переменную df
sessions_test_part = pd.read_csv('/datasets/sessions_project_test_part.csv')
```

In [15]:

```
# Выводим первые строки датафрейма на экран
sessions_test_part.head()
```

Out[15]:

	user_id	session_id	session_date	session_start_ts	install_date	session_number
0	3404844B53442747	B4901323BD537E45	2025-10-14	2025-10-14 19:28:49	2025-10-14	1

	user_id	session_id	session_date	session_start_ts	install_date	session_number
1	3A2BF4D364E62D89	216FC619308F8788	2025-10-14	2025-10-14 21:11:04	2025-10-14	1
2	79CDAE11E32B1597	EDFCE4AC1A504074	2025-10-14	2025-10-14 21:44:03	2025-10-14	1
3	D6AF8D78297A931F	CF0AC0EEDE92C690	2025-10-14	2025-10-14 19:07:55	2025-10-14	1
4	37E0CE723AE568E0	2E6ED45E8C86C4E9	2025-10-14	2025-10-14 15:39:44	2025-10-14	1

In [16]:

```
# Выводим информацию о датафрейме
sessions_test_part.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3130 entries, 0 to 3129
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   user_id                3130 non-null   object
1   session_id             3130 non-null   object
2   session_date           3130 non-null   object
3   session_start_ts       3130 non-null   object
4   install_date           3130 non-null   object
5   session_number         3130 non-null   int64
6   registration_flag      3130 non-null   int64
7   page_counter           3130 non-null   int64
8   region                 3130 non-null   object
9   device                 3130 non-null   object
10  test_group             3130 non-null   object
dtypes: int64(3), object(8)
memory usage: 269.1+ KB
```

In [17]:

```
# Считаем общее количество уникальных пользователей в каждой из экспериментальных групп
group_user = sessions_test_part.groupby('test_group')['user_id'].nunique()
print("\nОбщее количество уникальных пользователей в каждой из экспериментальных групп")
print(group_user)
```

```
Общее количество уникальных пользователей в каждой из экспериментальных групп для одного дня наблюдения:
test_group
A      1477
B      1466
Name: user_id, dtype: int64
```

In [18]:

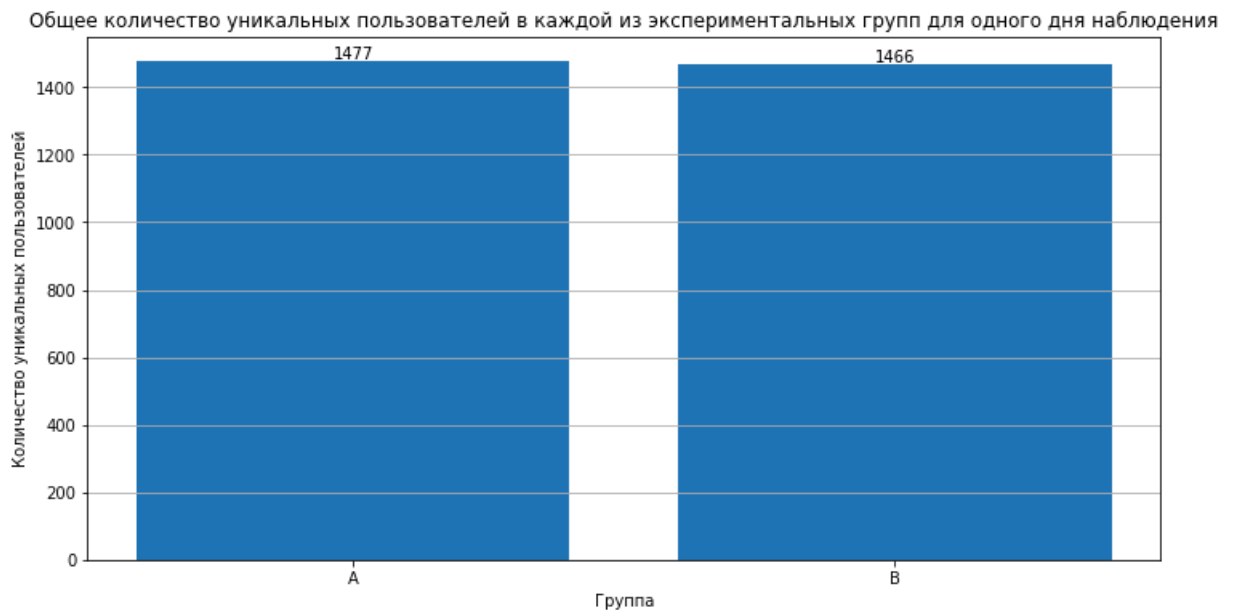
```
# Считаем процентную разницу в количестве пользователей в группах A и B
A = group_user.get('A')
B = group_user.get('B')
P = 100 * (A - B) / A
print(f"\nПроцентная разница в количестве пользователей в группах A и B: {P:.2f}%")
```

```
Процентная разница в количестве пользователей в группах A и B: 0.74%
```

In [19]:

```
# Строим график
plt.figure(figsize=(12, 6))
plt.bar(group_user.index, group_user.values)
# Настраиваем оформление графика
plt.title('Общее количество уникальных пользователей в каждой из экспериментальных групп')
plt.xlabel('Группа')
```

```
plt.ylabel('Количество уникальных пользователей')
# Добавляем подписи значений на столбцы графика
for i, v in enumerate(group_user.values):
    plt.text(i, v, str(v), ha='center', va='bottom')
# Добавляем сетку графика
plt.grid(axis='y')
# Выводим график
plt.show()
```



3.2. Проверка пересечений пользователей

Помимо проверки равенства количества пользователей в группах, полезно убедиться в том, что группы независимы. Для этого нужно убедиться, что никто из пользователей случайно не попал в обе группы одновременно.

- Рассчитайте количество пользователей, которые встречаются одновременно в группах A и B, или убедитесь, что таких нет.

In [20]:

```
# Считаем количество пользователей, которые встречаются одновременно в группах A и B
user_A = set(sessions_test_part.query("test_group == 'A'")['user_id'].unique())
user_B = set(sessions_test_part.query("test_group == 'B'")['user_id'].unique())
# Считаем пересечение
cross_user = user_A.intersection(user_B)
print(f"Количество пользователей, которые встречаются одновременно в группах A и B: {
```

Количество пользователей, которые встречаются одновременно в группах A и B: 0

3.3. Равномерность разделения пользователей по устройствам

Полезно также убедиться в том, что пользователи равномерно распределены по всем доступным категориальным переменным — типам устройств и регионам.

Постройте две диаграммы:

- доля каждого типа устройства для пользователей из группы A,
- доля каждого типа устройства для пользователей из группы B.

Постарайтесь добавить на диаграммы все необходимые подписи, пояснения и заголовки, которые позволят сделать вывод о том, совпадает ли распределение устройств в группах A

и B.

```
In [21]: # Считаем долю уникальных пользователей из группы A
user_A = sessions_test_part[sessions_test_part['test_group'] == 'A'][['user_id', 'device']]
# Считаем долю каждого типа устройства для пользователей из группы A
device_A = user_A['device'].value_counts(normalize=True)
# Строим график
plt.figure(figsize=(12, 6))
plt.bar(device_A.index, device_A.values)
# Настраиваем оформление графика
plt.title('Доля каждого типа устройства для пользователей из группы A')
plt.xlabel('Тип устройства')
plt.ylabel('Доля пользователей')
# Добавляем подписи значений на столбцы графика
for i, v in enumerate(device_A.values):
    plt.text(i, v, str(v), ha='center', va='bottom')
# Добавляем сетку графика
plt.grid(axis='y')
# Выводим график
plt.show()
print("Доля каждого типа устройства для пользователей из группы A:")
print(device_A)
```



Доля каждого типа устройства для пользователей из группы A:

```
Android    0.444144
PC          0.249831
iPhone     0.200406
Mac         0.105619
Name: device, dtype: float64
```

```
In [22]: # Считаем долю уникальных пользователей из группы B
user_B = sessions_test_part[sessions_test_part['test_group'] == 'B'][['user_id', 'device']]
# Считаем долю каждого типа устройства для пользователей из группы B
device_B = user_B['device'].value_counts(normalize=True)
# Строим график
plt.figure(figsize=(12, 6))
plt.bar(device_B.index, device_B.values)
# Настраиваем оформление графика
plt.title('Доля каждого типа устройства для пользователей из группы B')
plt.xlabel('Тип устройства')
plt.ylabel('Доля пользователей')
# Добавляем подписи значений на столбцы графика
```

```

for i, v in enumerate(device_B.values):
    plt.text(i, v, str(v), ha='center', va='bottom')
# Добавляем сетку графика
plt.grid(axis='y')
# Выводим график
plt.show()
print("Доля каждого типа устройства для пользователей из группы B:")
print(device_B)

```



Доля каждого типа устройства для пользователей из группы B:

```

Android    0.455662
PC         0.259891
iPhone     0.183492
Mac        0.100955
Name: device, dtype: float64

```

3.4. Равномерность распределения пользователей по регионам

Теперь убедитесь, что пользователи равномерно распределены по регионам.

Постройте две диаграммы:

- доля каждого региона для пользователей из группы A,
- доля каждого региона для пользователей из группы B.

Постарайтесь добавить на диаграммы все необходимые подписи, пояснения и заголовки, которые позволят сделать вывод о том, совпадает ли распределение регионов в группах A и B. Постарайтесь использовать другой тип диаграммы, не тот, что в прошлом задании.

In [23]:

```

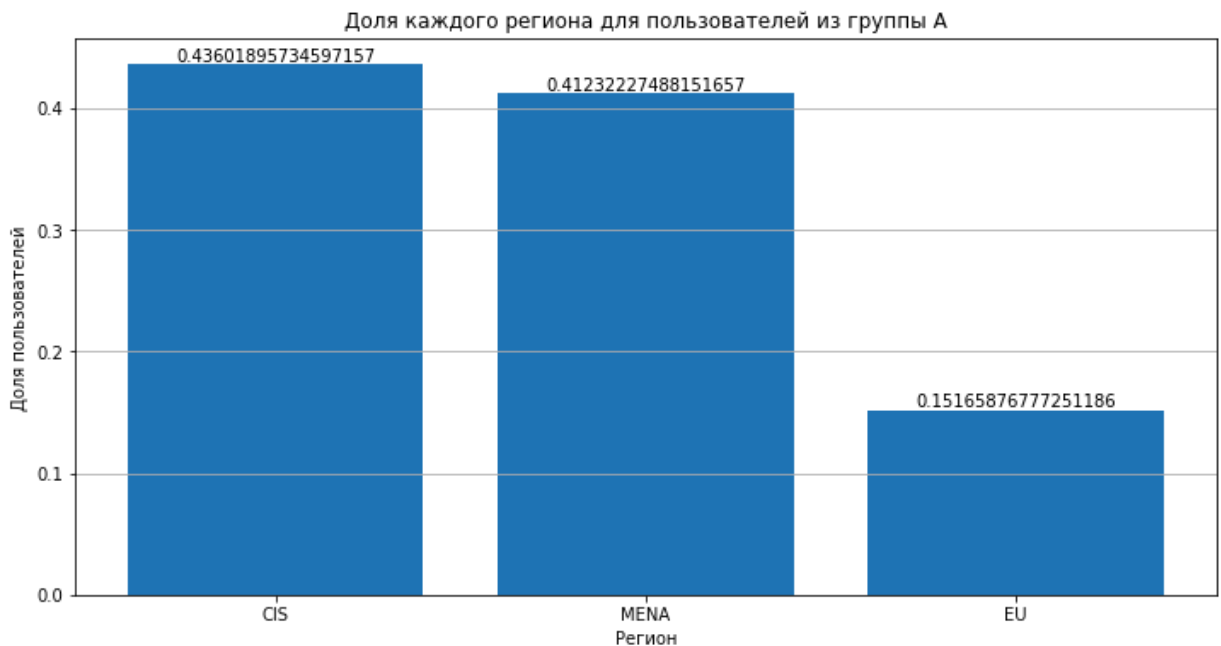
# Считаем долю каждого региона для пользователей из группы A
user_A = sessions_test_part[sessions_test_part['test_group'] == 'A'][['user_id', 'region']]
region_A = user_A['region'].value_counts(normalize=True)
# Строим график
plt.figure(figsize=(12, 6))
plt.bar(region_A.index, region_A.values)
# Настраиваем оформление графика
plt.title('Доля каждого региона для пользователей из группы A')
plt.xlabel('Регион')
plt.ylabel('Доля пользователей')
# Добавляем подписи значений на столбцы графика
for i, v in enumerate(region_A.values):

```

```

plt.text(i, v, str(v), ha='center', va='bottom')
# Добавляем сетку графика
plt.grid(axis='y')
# Выводим график
plt.show()
print("Доля каждого региона для пользователей из группы A:")
print(region_A)

```



Доля каждого региона для пользователей из группы A:

```

CIS      0.436019
MENA     0.412322
EU       0.151659
Name: region, dtype: float64

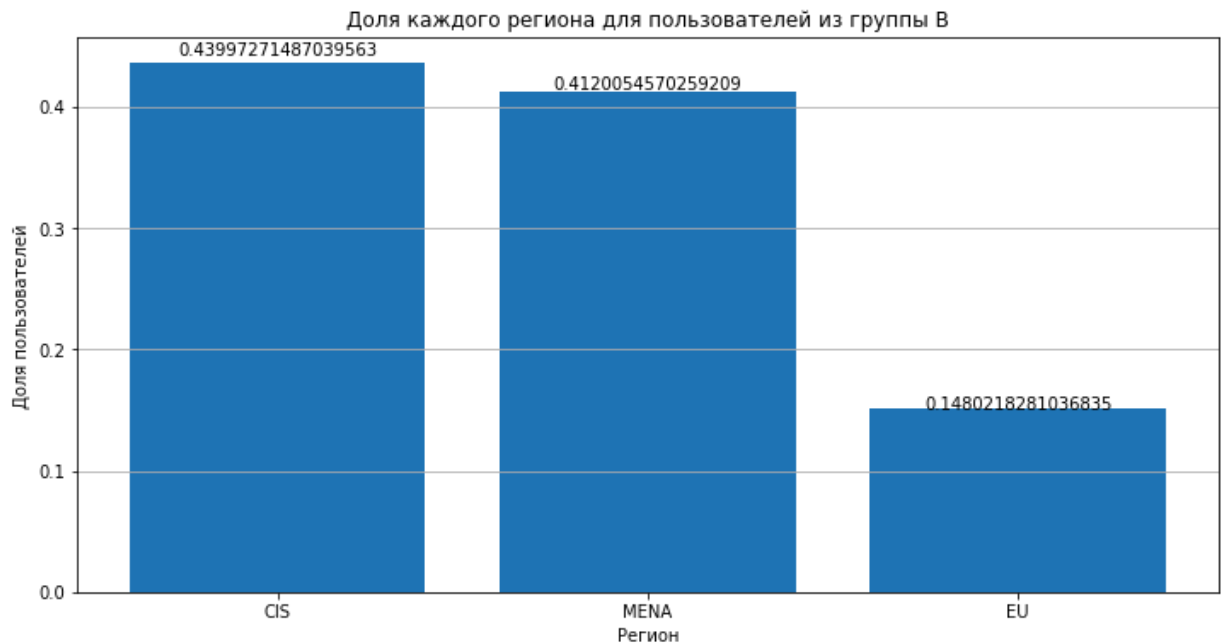
```

In [24]:

```

# Считаем долю каждого региона для пользователей из группы B
user_B = sessions_test_part[sessions_test_part['test_group'] == 'B'][['user_id', 'region']]
region_B = user_B['region'].value_counts(normalize=True)
# Строим график
plt.figure(figsize=(12, 6))
plt.bar(region_B.index, region_B.values)
# Настраиваем оформление графика
plt.title('Доля каждого региона для пользователей из группы B')
plt.xlabel('Регион')
plt.ylabel('Доля пользователей')
# Добавляем подписи значений на столбцы графика
for i, v in enumerate(region_B.values):
    plt.text(i, v, str(v), ha='center', va='bottom')
# Добавляем сетку графика
plt.grid(axis='y')
# Выводим график
plt.show()
print("Доля каждого региона для пользователей из группы B:")
print(region_B)

```



Доля каждого региона для пользователей из группы В:

CIS 0.439973

MENA 0.412005

EU 0.148022

Name: region, dtype: float64

3.5. Вывод после проверки A/B-теста

На основе проведённого анализа A/B-теста сформулируйте и запишите свои выводы. В выводе обязательно укажите:

- Было ли обнаружено различие в количестве пользователей в двух группах.
- Являются ли выборки независимыми. Было ли обнаружено пересечение пользователей из тестовой и контрольной групп.
- Сохраняется ли равномерное распределение пользователей тестовой и контрольной групп по категориальным переменным: устройствам и регионам.

Сделайте заключение: корректно ли проходит A/B-тест, или наблюдаются какие-либо нарушения.

1. По итогам расчета общего количества уникальных пользователей в каждой из экспериментальных групп для одного дня наблюдения в группах А и В примерно одинаковое количество, что составило в группе А - 1477, а в группе В - 1466, разница составляет в 11 пользователей. Процентная разница в количестве пользователей в группах А и В составила 0.74%. Из рассчитанных показателей видно, что большого разлия между группами отсутствует.
2. По итогам расчета количества пользователей, которые встречаются одновременно в группах А и В составило 0. Из рассчитанных показателей видно, что отсутствуют пользователи, котрые одновременно находятся в группе А и группе В. Выборки их независимы, пересечение отсутствует.
3. По итогам расчета доли каждого типа устройства для пользователей из группы А и группы В составляет примерно одинаковое количество, что составило в группе А:

- Android - 0.444144;

- PC - 0.249831;
 - iPhone - 0.200406;
 - Mac - 0.105619.
- в группе B:
- Android - 0.455662;
 - PC - 0.259891;
 - iPhone - 0.183492;
 - Mac - 0.100955.

По итогам расчета доли каждого региона для пользователей из группы A и группы B составляет примерно одинаковое количество, что составило в группе A:

- CIS - 0.436019;
 - MENA - 0.412322;
 - EU - 0.151659.
- в группе B:
- MENA - 0.412005;
 - CIS - 0.439973;
 - EU - 0.148022.

Заключение: A/B-тест проходит корректно, нарушений не наблюдается.

4. Проверка результатов A/B-теста

A/B-тест завершён, и у вас есть результаты за все дни проведения эксперимента. Необходимо убедиться в корректности теста и верно интерпретировать результаты.

4.1. Получение результатов теста и подсчёт основной метрики

- Считайте и сохраните в датафрейм `sessions_test` CSV-файл с историческими данными о сессиях пользователей `sessions_project_test.csv`.
- В датафрейме `sessions_test` создайте дополнительный столбец `good_session`. В него войдёт значение `1`, если за одну сессию было просмотрено 4 и более страниц, и значение `0`, если просмотрено меньше.

In [25]:

```
# Импортируем библиотеки
import pandas as pd
import numpy as np

# Загружаем библиотеки для визуализации данных
import matplotlib.pyplot as plt
import seaborn as sns
```

In [26]:

```
# Выгружаем данные в переменную df
sessions_test = pd.read_csv('/datasets/sessions_project_test.csv')
```

In [27]:

```
# Выводим первые строки датафрейма на экран
sessions_test_part.head()
```

Out[27]:

	user_id	session_id	session_date	session_start_ts	install_date	session_number
0	3404844B53442747	B4901323BD537E45	2025-10-14	2025-10-14 19:28:49	2025-10-14	1
1	3A2BF4D364E62D89	216FC619308F8788	2025-10-14	2025-10-14 21:11:04	2025-10-14	1
2	79CDAE11E32B1597	EDFCE4AC1A504074	2025-10-14	2025-10-14 21:44:03	2025-10-14	1
3	D6AF8D78297A931F	CF0AC0EEDE92C690	2025-10-14	2025-10-14 19:07:55	2025-10-14	1
4	37E0CE723AE568E0	2E6ED45E8C86C4E9	2025-10-14	2025-10-14 15:39:44	2025-10-14	1

In [28]:

```
# Выводим информацию о датафрейме
sessions_test_part.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3130 entries, 0 to 3129
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   user_id                3130 non-null   object
1   session_id             3130 non-null   object
2   session_date           3130 non-null   object
3   session_start_ts       3130 non-null   object
4   install_date           3130 non-null   object
5   session_number         3130 non-null   int64
6   registration_flag      3130 non-null   int64
7   page_counter           3130 non-null   int64
8   region                 3130 non-null   object
9   device                 3130 non-null   object
10  test_group             3130 non-null   object
dtypes: int64(3), object(8)
memory usage: 269.1+ KB
```

In [29]:

```
# Создаем дополнительный столбец good_session
sessions_test['good_session'] = (sessions_test['page_counter']>=4).astype(int)
print(sessions_test[['user_id', 'session_id', 'page_counter', 'good_session']].head(
```

	user_id	session_id	page_counter	good_session
0	6DAE3B3654DA738E	C69249E26E58F6E2	3	0
1	0A3FE5D1DD59110A	66D66D7C9F5181B7	2	0
2	2041F1D7AA740B88	50DE51D42215E74C	2	0
3	43D7585009168086	5763C0C353C22263	1	0
4	15AD68B14D62D88C	B1AD09F93C1053BC	2	0

4.2. Проверка корректности результатов теста

Прежде чем приступить к анализу ключевых продуктовых метрик, необходимо убедиться, что тест проведён корректно и вы будете сравнивать две сопоставимые группы.

- Рассчитайте количество уникальных сессий для каждого дня и обеих тестовых групп, используя группировку.
- Проверьте, что количество уникальных дневных сессий в двух выборках не различается или различия не статистически значимыми. Используйте статистический тест, который позволит сделать вывод о равенстве средних двух выборок.

- В качестве ответа выведите на экран полученное значение p-value и интерпретируйте его.

In [30]:

```
from scipy.stats import ttest_ind
# Считаем количество уникальных сессий для каждого дня и обеих тестовых групп, испол
daily_session = sessions_test.groupby(['session_date', 'test_group'])['session_id'].
# Считаем количество уникальных дневных сессий в двух выборках
daily_A = daily_session.query("test_group == 'A'")['session_id']
daily_B = daily_session.query("test_group == 'B'")['session_id']
# Используем статистический тест
stat_ttest, p_value_ttest = ttest_ind(daily_A, daily_B, equal_var=False)
print("Средняя выборка A:", daily_A.mean())
print("Средняя выборка B:", daily_B.mean())
print("t-статистика:", stat_ttest)
print("p_value_ttest:", p_value_ttest)
alpha = 0.05
if p_value_ttest < alpha:
    print("Средние в группах A и B статистически значимо различаются")
else:
    print("Нет оснований считать, что средние в группах A и B различаются")
```

Средняя выборка A: 2477.55

Средняя выборка B: 2522.7

t-статистика: -0.07802281548251855

p_value_ttest: 0.9382195854082714

Нет оснований считать, что средние в группах A и B различаются

4.3. Сравнение доли успешных сессий

Когда вы убедились, что количество сессий в обеих выборках не различалось, можно переходить к анализу ключевой метрики — доли успешных сессий.

Используйте созданный на первом шаге задания столбец `good_session` и рассчитайте долю успешных сессий для выборок A и B, а также разницу в этом показателе. Полученный вывод отобразите на экране.

In [31]:

```
# Считаем долю успешных сессий для выборок A и B
dol_session = sessions_test.groupby('test_group')['good_session'].mean()
dol_A = dol_session['A']
dol_B = dol_session['B']
# Считаем разницу
dif_session = dol_B - dol_A
print(dol_A)
print(dol_B)
print(dif_session)
```

0.3077233557344958

0.31828992745867524

0.010566571724179463

4.4. Насколько статистически значимо изменение ключевой метрики

На предыдущем шаге вы убедились, что количество успешных сессий в тестовой выборке примерно на 1.1% выше, чем в контрольной, но делать выводы только на основе этого значения будет некорректно. Для принятия решения всегда необходимо отвечать на вопрос: является ли это изменение статистически значимым.

- Используя статистический тест, рассчитайте, является ли изменение в метрике доли успешных сессий статистически значимым.
- Выведите на экран полученное значение p-value и свои выводы о статистической значимости. Напомним, что уровень значимости в эксперименте был выбран на уровне 0.05.

In [32]:

```
from statsmodels.stats.proportion import proportions_ztest
# Считаем количество успешных сессий в группе
session_count = sessions_test.groupby('test_group')['good_session'].sum()
# Считаем общее количество успешных сессий в группе
count_s = sessions_test.groupby('test_group')['good_session'].count()
# Берем значения
s_A_B = [session_count['A'], session_count['B']]
c_A_B = [count_s['A'], count_s['B']]
# Считаем Z-тест
stat_ztest, p_value_ztest = proportions_ztest(s_A_B, c_A_B)
print(p_value_ztest)
if p_value_ztest < 0.05:
    print("Разница значима")
else:
    print("Разница не значима")
```

0.0003149479976072246

Разница значима

4.5. Вывод по результатам A/B-эксперимента

На основе проведенного анализа результатов теста сформулируйте и запишите свои выводы для команды разработки приложения. В выводе обязательно укажите:

- Характеристики проведенного эксперимента, количество задействованных пользователей и длительность эксперимента.
- Повлияло ли внедрение нового алгоритма рекомендаций на рост ключевой метрики и как.
- Каким получилось значение p-value для оценки статистической значимости выявленного эффекта.
- Стоит ли внедрять нововведение в приложение.

1. По результатам проведенного эксперимента, поделенного на две группы А и В, составило почти одинаковые данные, анализ произведен корректно. Пересечений между группами не были обнаружены. Распределение по устройствам и регионам, так же дали почти одинаковые данные. Из этого можно предположить, что анализ проведен корректно и выборки одинаковы.
2. Внедрение нового алгоритма рекомендаций на рост ключевой метрики не повлияло. Разница между группой А и В составила в 0.010566571724179463.
3. Значение p-value для оценки статистической значимости выявленного эффекта получилось 0.0003149479976072246.
4. Внедрять нововведение в приложение по выводу проделанного анализа не стоит, так как новый алгоритм показал небольшой рост ключевой метрики.

Итоговый вывод: результат эксперимента анализа положительный, для статистического анализа недостаточно данных. Следует увеличить длительность, расширить выборку. Так же провести больше анализа по другим показателям, двух показателей для расчета показалось недостаточным.